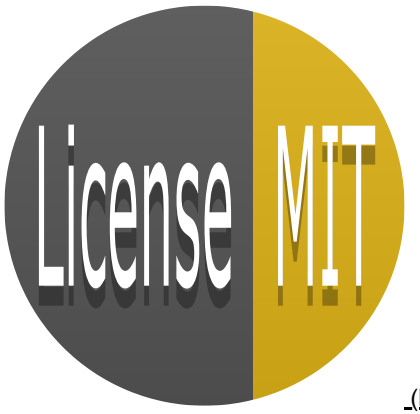
□ Advanced Personal Finance Tracker

> *An AI-powered personal finance management system for individuals and small businesses with intelligent insights, predictions, and automated financial analysis.*



(https://opensource.org/licenses/MIT)

(https://www.typescriptlang.org/)

(https://nextjs.org/)

□ Table of Contents

- Overview
- Tech Stack
- Key Features
- 15 AI Features
- Development Phases
- Project Structure
- Getting Started
- Environment Variables
- Database Schema

- API Documentation
- Deployment
- Contributing
- License

□ Overview

A comprehensive finance tracker that combines traditional financial management with cutting-edge AI capabilities. Designed for both individuals and small businesses, featuring local AI processing via Ollama for privacy-first financial insights.

Target Audience: Individuals, families, freelancers, and small business owners
Business Model: Freemium (Free tier with premium AI features)
Deployment: Self-hosted with Docker

□ Tech Stack

Core Technologies

- □ **Frontend**: Next.js 14 (App Router), TypeScript, TailwindCSS
- □ **Backend**: Express.js, TypeScript
- □ **Authentication**: Clerk
- □ **Database**: PostgreSQL
- □ **ORM**: Drizzle ORM
- □ **Validation**: Zod
- □ **State Management**: Zustand
- □ **AI/ML**: LangChain + Ollama (local LLMs)
- □ **Vector Database**: Quadrant (for RAG)

Additional Libraries

File Processing

- □ `papaparse` - CSV parsing
- □ `xlsx` - Excel file handling
- □ `multer` - File uploads

Email Integration

- □ `node-imap` - Email fetching
- □ `mailparser` - Email parsing
- □ `cheerio` - HTML parsing

AI/ML Enhancements

- □ `@xenova/transformers` - Client-side embeddings
- □ `pdf-parse` - PDF statement parsing
- □ `tesseract.js` - OCR for receipt scanning

Visualization

- □ `recharts` - Data visualization
- □ `react-financial-charts` - Financial charts

UI Components

- □ `shadcn/ui` - UI component library
- □ `react-hook-form` - Form management
- □ `date-fns` - Date manipulation
- □ `numeral` - Number formatting

Background Processing

- □ `bullmq` - Job queue for AI processing
- □ `node-cron` - Scheduled tasks

Testing

- □ `vitest` - Unit testing
- □ `playwright` - E2E testing
- □ `msw` - API mocking

DevOps

- □ `docker` - Containerization
- □ `docker-compose` - Local development
- □ `nginx` - Reverse proxy

□ Key Features

Core Financial Management

- [ ] Multi-account management (checking, savings, credit cards, cash)
- [ ] Manual transaction entry with natural language support
- [ ] CSV/Excel import for bulk transactions
- [ ] Email receipt parsing and auto-import
- [ ] Category and tag management
- [ ] Recurring transaction tracking

- [ ] Multi-currency support with auto-conversion
- [ ] Budget creation and tracking
- [ ] Financial goal setting and monitoring
- [ ] Bill payment reminders

Reporting & Analytics

- [ ] Income vs Expense reports
- [ ] Cash flow analysis
- [ ] Category breakdown and trends
- [ ] Tax reports and deduction tracking
- [ ] Profit & Loss statements (for businesses)
- [ ] Custom date range filtering
- [ ] Export to PDF/Excel

Business-Specific Features

- [ ] Client/vendor tracking
- [ ] Invoice management
- [ ] Business expense categorization
- [ ] Tax optimization suggestions
- [ ] Multi-entity support

User Experience

- [ ] Dark mode support
- [ ] Mobile-responsive design
- [ ] Keyboard shortcuts
- [ ] Advanced search and filtering
- [ ] Transaction attachments (receipts, invoices)
- [ ] Custom dashboard widgets

□ 15 AI Features

Free Tier AI Features

- [ ] **1. Smart Transaction Categorization** - Auto-categorize transactions using AI
- [ ] **2. Natural Language Transaction Entry** - "Spent $50 on groceries at Whole Foods"
- [ ] **4. Spending Pattern Analysis** - Identify spending habits (limited in free tier)
- [ ] **7. Anomaly Detection** - Detect unusual transactions or fraud
- [ ] **8. Smart Bill Reminders** - Predict upcoming bills based on patterns

- [ ] **11. Multi-Currency Intelligence** - Handle forex with rate explanations
- [ ] **12. Merchant Recognition & Enrichment** - Standardize merchant names, add logos
- [ ] **14. Smart Receipt Search** - Natural language search (limited searches/month)

Premium AI Features

- [ ] **3. Intelligent Email Receipt Parser** - Auto-extract transactions from emails
- [ ] **5. Predictive Budget Alerts** - Forecast budget overruns
- [ ] **6. AI Financial Advisor Chat** - Ask questions about your finances
- [ ] **9. Tax Optimization Suggestions** - Identify tax-deductible expenses
- [ ] **10. Financial Goal Coaching** - Personalized advice to reach goals
- [ ] **13. Cash Flow Forecasting** - 3-month predictions
- [ ] **15. Automated Financial Reports** - AI-generated monthly summaries

---

□ Development Phases

□ Phase 1: Foundation & Core Features (4-6 weeks)

Week 1-2: Project Setup & Authentication

- [ ] Initialize Next.js + Express monorepo
- [ ] Configure TypeScript, ESLint, Prettier
- [ ] Set up PostgreSQL database
- [ ] Configure Drizzle ORM with schema
- [ ] Implement Clerk authentication
- [ ] Create base UI components (shadcn/ui)
- [ ] Docker setup for local development
- [ ] CI/CD pipeline configuration

Week 3-4: Core Finance Features

- [ ] Design database schema (accounts, transactions, categories, budgets)
- [ ] Implement manual transaction CRUD
- [ ] Build account management system
- [ ] Create category management
- [ ] Develop simple dashboard with summary stats
- [ ] Add basic filtering and search

Week 5-6: CSV/Excel Import

- [ ] File upload system implementation
- [ ] CSV parser (papaparse)
- [ ] Excel parser (xlsx)

- [ ] Transaction mapping interface
- [ ] Import preview and validation
- [ ] Bulk import error handling

Phase 1 Deliverables:

- [ ] Working authentication system
- [ ] Manual transaction entry
- [ ] CSV/Excel import functionality
- [ ] Basic dashboard
- [ ] Account management

□ Phase 2: AI Infrastructure & Basic Intelligence (4-6 weeks)

Week 7-8: AI Setup

- [ ] Ollama installation and configuration
- [ ] Model selection (Mistral/Llama3)
- [ ] LangChain integration
- [ ] Quadrant vector database setup
- [ ] Embedding pipeline creation
- [ ] Basic RAG implementation
- [ ] AI response caching system

Week 9-10: First AI Features

- [ ] AI Feature #1: Smart Transaction Categorization
- [ ] AI Feature #2: Natural Language Transaction Entry
- [ ] AI Feature #7: Anomaly Detection
- [ ] AI Feature #12: Merchant Recognition
- [ ] AI settings and preferences UI

Week 11-12: Email Integration

- [ ] Email connection setup (IMAP/OAuth2)
- [ ] Email parser implementation
- [ ] AI Feature #3: Intelligent Email Receipt Parser
- [ ] Email settings UI
- [ ] Background email polling job
- [ ] Email processing queue

Phase 2 Deliverables:

- [ ] Ollama + LangChain operational
- [ ] 4 AI features functional
- [ ] Email parsing system
- [ ] Vector database with embeddings

□ Phase 3: Advanced Analytics & Reporting (4-5 weeks)

Week 13-14: Budgeting & Goals

- [ ] Budget creation and management
- [ ] Budget tracking dashboard
- [ ] Financial goal system
- [ ] Goal progress tracking
- [ ] AI Feature #8: Smart Bill Reminders
- [ ] AI Feature #10: Financial Goal Coaching
- [ ] Budget vs actual visualization

Week 15-16: Reports & Visualization

- [ ] Income vs Expense reports
- [ ] Cash flow analysis charts
- [ ] Category breakdown reports
- [ ] Tax reports and summaries
- [ ] P&L statements for businesses
- [ ] AI Feature #9: Tax Optimization Suggestions
- [ ] Report export (PDF/Excel)

Week 17: Analytics Enhancement

- [ ] AI Feature #4: Spending Pattern Analysis
- [ ] AI Feature #13: Cash Flow Forecasting
- [ ] Advanced data visualization
- [ ] Trend analysis charts
- [ ] Custom report builder

Phase 3 Deliverables:

- [ ] Complete budgeting system
- [ ] Financial goals tracking
- [ ] Comprehensive reporting suite
- [ ] 4 additional AI features

□ Phase 4: Premium AI Features & Monetization (4-5 weeks)

Week 18-19: Conversational AI

- ☐ AI Feature #6: AI Financial Advisor Chat
- ☐ Chat interface with history
- ☐ Context-aware responses
- ☐ Conversation memory management
- ☐ Chat export functionality
- ☐ Rate limiting for free tier

Week 20-21: Predictive Features

- ☐ AI Feature #5: Predictive Budget Alerts
- ☐ AI Feature #15: Automated Financial Reports
- ☐ AI Feature #14: Smart Receipt Search (semantic)
- ☐ AI Feature #11: Multi-Currency Intelligence
- ☐ Notification system for predictions

Week 22: Subscription System

- ☐ Pricing page implementation
- ☐ Payment integration (Stripe/Paddle)
- ☐ Feature gating (free vs premium)
- ☐ Usage tracking and limits
- ☐ Subscription management UI
- ☐ Billing portal
- ☐ Usage analytics dashboard

Phase 4 Deliverables:

- ☐ AI chat assistant
- ☐ All 15 AI features complete
- ☐ Payment and subscription system
- ☐ Freemium model fully implemented

□ Phase 5: Polish, Performance & Launch (3-4 weeks)

Week 23: Performance Optimization

- ☐ Database query optimization
- ☐ AI response caching
- ☐ Background job processing (BullMQ)

- [ ] Image optimization
- [ ] Code splitting and lazy loading
- [ ] Bundle size optimization
- [ ] Performance monitoring setup

Week 24: Testing & Quality

- [ ] Unit tests (Vitest) - target 80% coverage
- [ ] Integration tests
- [ ] E2E tests (Playwright)
- [ ] AI feature testing
- [ ] Load testing
- [ ] Security audit
- [ ] Accessibility audit (WCAG 2.1)

Week 25: Documentation & Deployment

- [ ] User documentation
- [ ] API documentation
- [ ] Self-hosting guide
- [ ] Docker production setup
- [ ] Nginx configuration
- [ ] Database migration scripts
- [ ] Backup and restore procedures
- [ ] Monitoring setup (Sentry)

Week 26: Launch Preparation

- [ ] Beta testing program
- [ ] Bug fixes from beta
- [ ] Analytics setup (PostHog/Plausible)
- [ ] Error tracking (Sentry)
- [ ] Marketing site finalization
- [ ] Launch checklist completion
- [ ] Production deployment

Phase 5 Deliverables:

- [ ] Production-ready application
- [ ] Comprehensive test coverage
- [ ] Complete documentation
- [ ] Deployment scripts and guides
- [ ] Monitoring and analytics

□ Application Pages

Public Pages

- [ ] `/` - Landing page
- [ ] `/pricing` - Pricing tiers
- [ ] `/features` - Feature showcase
- [ ] `/about` - About page
- [ ] `/contact` - Contact form
- [ ] `/login` - Login page
- [ ] `/signup` - Registration

Dashboard Pages

- [ ] `/dashboard` - Main overview
- [ ] `/transactions` - Transaction list
- [ ] `/transactions/add` - Add transaction
- [ ] `/transactions/import` - Import CSV/Excel
- [ ] `/transactions/[id]` - Transaction details
- [ ] `/accounts` - Account management
- [ ] `/accounts/add` - Add account
- [ ] `/accounts/[id]` - Account details
- [ ] `/budgets` - Budget management
- [ ] `/budgets/create` - Create budget
- [ ] `/budgets/[id]` - Budget tracking
- [ ] `/goals` - Financial goals
- [ ] `/goals/create` - Create goal
- [ ] `/reports` - Reports hub
- [ ] `/reports/income-expense` - Income vs Expense
- [ ] `/reports/cash-flow` - Cash flow
- [ ] `/reports/tax` - Tax reports
- [ ] `/reports/profit-loss` - P&L
- [ ] `/categories` - Category management
- [ ] `/tags` - Tag management
- [ ] `/recurring` - Recurring transactions

AI-Powered Pages

- [ ] `/ai-assistant` - AI chat interface
- [ ] `/ai-insights` - AI insights dashboard

- [ ] `/ai-predictions` - Spending forecasts
- [ ] `/ai-recommendations` - Personalized tips
- [ ] `/ai-anomalies` - Anomaly alerts

Settings Pages

- [ ] `/settings` - General settings
- [ ] `/settings/profile` - User profile
- [ ] `/settings/security` - Security settings
- [ ] `/settings/notifications` - Notifications
- [ ] `/settings/email-integration` - Email setup
- [ ] `/settings/export` - Data export
- [ ] `/settings/subscription` - Subscription
- [ ] `/admin` - Admin dashboard

□ Getting Started

Prerequisites

```
- Node.js 18+ and npm/pnpm/yarn
- PostgreSQL 15+
- Docker and Docker Compose
- Ollama installed locally
```

Installation

```
# Clone the repository
git clone <repository-url>
cd finance-tracker

# Install dependencies
npm install

# Set up environment variables
cp .env.example .env
# Edit .env with your configuration

# Start Docker services (PostgreSQL, Quadrant)
docker-compose up -d

# Run database migrations
npm run db:migrate

# Seed initial data (optional)
npm run db:seed

# Pull Ollama model
ollama pull mistral

# Start development servers
npm run dev
```

The application will be available at:

- Frontend: http://localhost:3000
- Backend API: http://localhost:5000

□ Environment Variables

Create a `.env` file in the root directory:

```
# Database
DATABASE_URL=postgresql://user:password@localhost:5432/finance_tracker

# Clerk Authentication
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_xxx
CLERK_SECRET_KEY=sk_test_xxx

# Ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=mistral

# Quadrant Vector DB
QUADRANT_URL=http://localhost:6333
QUADRANT_API_KEY=your_api_key

# Email Integration (Optional)
EMAIL_IMAP_HOST=imap.gmail.com
EMAIL_IMAP_PORT=993
EMAIL_USER=your-email@gmail.com
EMAIL_PASSWORD=your-app-password

# Stripe (for payments)
STRIPE_PUBLIC_KEY=pk_test_xxx
STRIPE_SECRET_KEY=sk_test_xxx
STRIPE_WEBHOOK_SECRET=whsec_xxx

# App Configuration
NODE_ENV=development
API_URL=http://localhost:5000
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

□ Database Schema

Key tables:

- `users` - User accounts (managed by Clerk)
- `accounts` - Financial accounts (bank, credit card, cash)
- `transactions` - All financial transactions
- `categories` - Transaction categories
- `tags` - Transaction tags
- `budgets` - Budget definitions

- `goals` - Financial goals
- `recurring_transactions` - Recurring transaction patterns
- `email_receipts` - Parsed email receipts
- `ai_insights` - Generated AI insights
- `ai_conversations` - Chat history

Full schema documentation: docs/database-schema.md (docs/database-schema.md)

□ API Documentation

Authentication

All API endpoints require authentication via Clerk session token.

Core Endpoints

```
// Transactions
GET    /api/transactions
POST   /api/transactions
GET    /api/transactions/:id
PUT    /api/transactions/:id
DELETE /api/transactions/:id

// Accounts
GET    /api/accounts
POST   /api/accounts
PUT    /api/accounts/:id
DELETE /api/accounts/:id

// Budgets
GET    /api/budgets
POST   /api/budgets
GET    /api/budgets/:id
PUT    /api/budgets/:id

// AI Features
POST   /api/ai/categorize
POST   /api/ai/chat
GET    /api/ai/insights
POST   /api/ai/predict-budget
GET    /api/ai/anomalies

// Import
POST   /api/import/csv
POST   /api/import/excel
POST   /api/import/email
```

Full API documentation: docs/api-reference.md (docs/api-reference.md)

□ Project Structure

```
finance-tracker/
├── apps/
│   ├── web/                 # Next.js frontend
│   │   ├── app/            # App router pages
│   │   ├── components/     # React components
│   │   ├── lib/            # Utilities
│   │   └── styles/         # Global styles
│   └── api/                # Express backend
│       ├── src/
│       │   ├── routes/     # API routes
│       │   ├── services/   # Business logic
│       │   ├── ai/         # AI features
│       │   └── db/         # Database
│       └── tests/
├── packages/
│   ├── db/                 # Drizzle schema & migrations
│   ├── ui/                 # Shared UI components
│   └── types/              # Shared TypeScript types
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
├── docs/
│   ├── api-reference.md
│   ├── database-schema.md
│   └── deployment.md
└── scripts/
    ├── setup.sh
    └── migrate.sh
```

□ Deployment

Docker Deployment

```
# Build production images
docker-compose -f docker-compose.prod.yml build

# Start services
docker-compose -f docker-compose.prod.yml up -d

# View logs
docker-compose logs -f
```

Manual Deployment

See detailed deployment guide: docs/deployment.md (docs/deployment.md)

□ Testing

```
# Run all tests
npm test

# Run with coverage
npm run test:coverage

# Run E2E tests
npm run test:e2e

# Run specific test file
npm test -- transactions.test.ts
```

□ Performance Targets

- ☐ Page load time: < 2 seconds
- ☐ AI response time: < 1 second (cached)
- ☐ Database query time: < 100ms (p95)
- ☐ Uptime: 99.9%
- ☐ Test coverage: > 80%
- ☐ Lighthouse score: > 90

□ Security

- [ ] Authentication via Clerk with MFA support
- [ ] API rate limiting
- [ ] CORS configuration
- [ ] SQL injection prevention (Drizzle ORM)
- [ ] XSS protection
- [ ] CSRF tokens
- [ ] Encrypted sensitive data at rest
- [ ] Secure HTTP headers (helmet.js)
- [ ] Regular dependency updates
- [ ] Security audit before launch

□ Contributing

Contributions are welcome! Please read CONTRIBUTING.md (CONTRIBUTING.md) for details.

1. Fork the repository
2. Create your feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

□ License

This project is licensed under the MIT License - see the LICENSE (LICENSE) file for details.

□ Acknowledgments

- Next.js (https://nextjs.org/) - React framework
- Ollama (https://ollama.ai/) - Local LLM runtime
- LangChain (https://www.langchain.com/) - LLM orchestration
- Clerk (https://clerk.com/) - Authentication
- Drizzle ORM (https://orm.drizzle.team/) - Type-safe ORM
- shadcn/ui (https://ui.shadcn.com/) - UI components

□ Support

- Documentation: docs/ (docs/)
- Issues: GitHub Issues (https://github.com/your-username/finance-tracker/issues)
- Discussions: GitHub Discussions (https://github.com/your-username/finance-tracker/discussions)

□ Roadmap

Version 1.0 (Current Plan)

- □ All 15 AI features
- □ Core financial management
- □ Freemium model

Version 2.0 (Future)

- ☐ Mobile apps (React Native)
- ☐ Bank API integration (Plaid)
- ☐ Receipt OCR scanning
- ☐ Investment tracking
- ☐ Collaborative budgets (family/team)
- ☐ Advanced tax optimization
- ☐ Multi-language support
- ☐ White-label solution

Built with ❤ for financial freedom